

Київський національний університет імені Тараса Шевченка
Факультет комп'ютерних наук та кібернетики

ЗВІТ
з лабораторної роботи
з дисципліни
«Обчислювальна геометрія та комп'ютерна графіка»

на тему:
**«Найкоротший шлях на множині полігональних
перешкод у 2D»**

Виконав:
студент групи ТК-32
Сікора Віктор

Київ — 2026

ЗМІСТ

1. Мета роботи	3
2. Теоретичні відомості	3
2.1. Постановка задачі	3
2.2. Геометричні примітиви	4
2.3. Граф видимості	5
2.4. Алгоритм Дейкстри	6
2.5. Аналіз складності	7
3. Структура програми	8
4. Реалізація: ключові фрагменти коду	9
4.1. Геометричні функції	9
4.2. Перевірка видимості	10
4.3. Побудова графа видимості	11
4.4. Алгоритм Дейкстри	12
4.5. GUI-інтерфейс	13
5. Тестування	14
5.1. Автоматичні тести	14
5.2. Приклади роботи програми	15
6. Висновки	22
Список використаних джерел	23

1. МЕТА РОБОТИ

Метою лабораторної роботи є вивчення та практична реалізація алгоритму пошуку найкоротшого евклідового шляху у двовимірному середовищі з полігональними перешкодами. Задача розв'язується двоетапно: спочатку будується граф видимості, потім до нього застосовується алгоритм Дейкстри.

- Реалізувати геометричні примітиви: векторний добуток, перевірка перетину відрізків, належність точки многокутнику.
- Побудувати граф видимості для довільної множини опуклих і неопуклих перешкод.
- Знайти найкоротший шлях алгоритмом Дейкстри з бінарною купою.
- Розробити інтерактивний GUI (matplotlib) для введення та візуалізації задачі.
- Покрити обчислювальну логіку автоматичними тестами.

2. ТЕОРЕТИЧНІ ВІДОМОСТІ

2.1 Постановка задачі

Дано множину з n полігональних перешкод із сумарною кількістю вершин n та дві точки: S (старт) і T (ціль), що лежать поза перешкодами. Потрібно знайти евклідово-найкоротший шлях від S до T , що не перетинає жодну перешкоду.

Ключовий теоретичний результат: оптимальний шлях завжди є ламаною, вершини якої — це S , T та деяка підмножина вершин перешкод. Тому достатньо шукати найкоротший шлях у графі видимості.

$$\sum |p_{i-1} p_i| \rightarrow \min, \text{ кожен відрізок } p_{i-1} p_i \cap \text{int}(P_j) = \emptyset$$

де $\text{int}(P_j)$ — внутрішність j -ї перешкоди.

2.2 Геометричні примітиви

2.2.1 Знаковий векторний добуток

Основним інструментом планарної обчислювальної геометрії є знаковий векторний добуток:

$$\text{cross}(O, A, B) = (A - O) \times (B - O) = (A_x - O_x)(B_y - O_y) - (A_y - O_y)(B_x - O_x)$$

- $\text{cross} > 0$ — точка B зліва від напрямку OA .
- $\text{cross} < 0$ — точка B справа від напрямку OA .
- $\text{cross} = 0$ — точки O, A, B колінеарні.

2.2.2 Трансверсальне перетинання відрізків

Два відрізки $p_1 p_2$ та $p_3 p_4$ перетинаються трансверсально, якщо точки одного лежать по різні боки прямої, що містить інший:

$\text{cross}(p_3, p_4, p_1) \cdot \text{cross}(p_3, p_4, p_2) < 0 \wedge \text{cross}(p_1, p_2, p_3) \cdot \text{cross}(p_1, p_2, p_4) < 0$
 Дотик у спільній вершині полігона не вважається перетином — це критично для правильного формування графа видимості.

2.2.3 Належність точки многокутнику (Ray Casting)

З точки p пускається горизонтальний промінь вправо. Кількість перетинів з ребрами многокутника: непарна — точка всередині, парна — зовні. Граничний випадок (точка на межі) перевіряється окремою функцією.

2.3 Граф видимості

Граф видимості $G = (V, E)$ будується за такими правилами:

- $V = \{S, T\} \cup \{\text{усі вершини перешкод}\}$, $|V| = n + 2$.
- Ребро $(u, v) \in E$, якщо відрізок uv не перетинає трансверсально жодного ребра перешкоди і середина uv не лежить всередині жодного полігона.
- Вага ребра — евклідова відстань $|uv|$.

Наївна побудова: $O(N^2)$ пар $\times O(n)$ перевірок = $O(N^2 \cdot n)$. Оптимальний алгоритм (Lee & Preparata, 1984) досягає $O(n^2 \log n)$.

Існують алгоритми $O(E + n \log n)$ (Ghosh & Mount, 1991), де E — кількість ребер у фінальному графі видимості.

2.4 Алгоритм Дейкстри

Після побудови графа видимості задача зводиться до пошуку найкоротшого шляху у зваженому графі з невід'ємними вагами — класичне застосування алгоритму Дейкстри.

Алгоритм (версія з бінарною купою):

- $\text{dist}[S] = 0$; $\text{dist}[v] = \infty$ для всіх $v \neq S$.
- Черга з пріоритетом: $Q \leftarrow \{(0, S)\}$.
- Поки Q не порожня: витягти (d, u) з мінімальним d .
- Для кожного сусіда v : якщо $d + w(u, v) < \text{dist}[v]$, оновити $\text{dist}[v]$ і додати $(\text{dist}[v], v)$ до Q .
- Зупинитись при витяганні T ; відновити шлях через масив $\text{parent}[]$.

Складність з бінарною купою: $O((V + E) \log V)$. Для графа видимості $E = O(N^2)$, тому $O(N^2 \log N)$.

2.5 Аналіз складності

Етап	Складність	Коментар
Перевірка одного ребра (u, v)	$O(n)$	Перебір усіх ребер перешкод
Побудова графа видимості (наїв)	$O(N^2 \cdot n)$	$N = n + 2$ вершин

Етап	Складність	Коментар
Побудова графа (Lee & Preparata)	$O(n^2 \log n)$	Кутове розгортання
Алгоритм Дейкстри	$O(N^2 \log N)$	Бінарна купа, $E = O(N^2)$
Загальна (наїв реалізація)	$O(n^3)$	Домінує побудова графа
Загальна (оптимальна)	$O(n^2 \log n)$	Lee & Preparata, 1984

3. СТРУКТУРА ПРОГРАМИ

Програма складається з трьох модулів:

Файл	Призначення
main.py	Геометричні примітиви, побудова графа видимості, алгоритм Дейкстри, GUI на matplotlib
test_algo.py	Автоматичні юніт-тести обчислювальної логіки (без графічного дисплея)
test_visual.py	Безголова генерація PNG-ілюстрацій для трьох прикладів

Внутрішня структура main.py поділена на чотири блоки:

- Геометричні примітиви — cross_v, segments_cross_properly, point_on_segment, point_in_polygon_strict, visible, euclidean.
- Граф видимості та Дейкстра — build_visibility_graph, dijkstra, find_shortest_path.
- GUI — клас ShortestPathApp (matplotlib widgets).
- Точка входу — функція main().

4. РЕАЛІЗАЦІЯ: КЛЮЧОВІ ФРАГМЕНТИ КОДУ

4.1 Геометричні функції

Базисом усієї геометричної логіки є знаковий векторний добуток. Реалізується однорядковою формулою:

main.py — cross_v()

```
def cross_v(o, a, b):
    return (a[0] - o[0]) * (b[1] - o[1]) \
        - (a[1] - o[1]) * (b[0] - o[0])
```

На основі cross_v визначається трансверсальне перетинання відрізків. Дотик у спільній вершині (нормальна ситуація для ребер графа видимості) не вважається перетином — це реалізується суворою нерівністю (EPS-порогом):

main.py — segments_cross_properly()

```
def segments_cross_properly(p1, p2, p3, p4):
    d1 = cross_v(p3, p4, p1); d2 = cross_v(p3, p4, p2)
    d3 = cross_v(p1, p2, p3); d4 = cross_v(p1, p2, p4)
    if ((d1 > EPS and d2 < -EPS) or (d1 < -EPS and d2 > EPS)) and
        ((d3 > EPS and d4 < -EPS) or (d3 < -EPS and d4 > EPS)):
        return True
    return False
```

Алгоритм Ray Casting для перевірки належності точки многокутнику. Граничний випадок (точка на межі) повертає False — точки на ребрі перешкоди вважаються допустимими позиціями для S/T:

main.py – point_in_polygon_strict() (фрагмент)

```
def point_in_polygon_strict(p, polygon):
    if point_on_polygon_boundary(p, polygon):
        return False # межа не є «всередині»
    x, y = p
    inside = False
    j = len(polygon) - 1
    for i in range(len(polygon)):
        xi, yi = polygon[i]; xj, yj = polygon[j]
        if (yi > y) != (yj > y):
            x_int = (xj - xi) * (y - yi) / (yj - yi) + xi
            if x < x_int:
                inside = not inside
        j = i
    return inside
```

4.2 Перевірка видимості

Функція visible(u, v, polygons) виконує дві незалежні перевірки:

- Перевірка 1: відрізок uv не перетинає трансверсально жодне ребро жодного полігона.
- Перевірка 2: середина відрізка uv не лежить всередині жодного полігона (закриває випадок діагоналі опуклого полігона).

main.py – visible()

```
def visible(u, v, polygons):
    for poly in polygons:
        m = len(poly)
        for i in range(m):
            a, b = poly[i], poly[(i + 1) % m]
            if segments_cross_properly(u, v, a, b):
                return False # перевірка 1
    mid = ((u[0]+v[0])*0.5, (u[1]+v[1])*0.5)
    for poly in polygons:
        if point_in_polygon_strict(mid, poly):
            return False # перевірка 2
    return True
```

4.3 Побудова графа видимості

Функція build_visibility_graph перебирає всі пари вершин і додає ребро тоді й тільки тоді, коли visible() повертає True. Складність: $O(N^2 \cdot n)$.

main.py – build_visibility_graph()

```
def build_visibility_graph(points, polygons):
    n = len(points)
    adj = [[] for _ in range(n)]
    edges = []
    for i in range(n):
        for j in range(i + 1, n):
            if visible(points[i], points[j], polygons):
                d = euclidean(points[i], points[j])
                adj[i].append((j, d))
                adj[j].append((i, d))
                edges.append((i, j))
    return adj, edges
```

4.4 Алгоритм Дейкстри

Реалізація використовує модуль `heapq` (бінарна мінімальна купа). «Ледача» версія: застарілі записи ігноруються при витяганні (if $d > \text{dist}[u]$: `continue`). Складність: $O(N^2 \log N)$.

`main.py` – `dijkstra()`

```
def dijkstra(adj, source, target):
    dist = [math.inf] * len(adj)
    parent = [-1] * len(adj)
    dist[source] = 0.0
    heap = [(0.0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue # застарілий запис – пропустити
        if u == target:
            break
        for v, w in adj[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                parent[v] = u
                heapq.heappush(heap, (nd, v))
    if math.isinf(dist[target]):
        return None, math.inf
    path, cur = [], target # відновлення шляху
    while cur != -1:
        path.append(cur); cur = parent[cur]
    path.reverse()
    return path, dist[target]
```

4.5 GUI-інтерфейс

Клас `ShortestPathApp` побудований на `matplotlib.widgets`. Три режими введення (`RadioButtons`), п'ять кнопок дій, рядок статусу. Нижче — фрагмент `_compute()`, що запускає весь pipeline:

`main.py` – `_compute()` (фрагмент)

```

def _compute(self, event):
    if self.S is None or self.T is None:
        self._set_status("Помилка: задайте S i T.")
        return
    for i, poly in enumerate(self.polygons):
        if point_in_polygon_strict(self.S, poly):
            self._set_status(f"Помилка: S всередині перешкоди #{i+1}.")
            return
    path, length, pts, edges = find_shortest_path(
        self.S, self.T, self.polygons)
    self.all_points, self.vg_edges = pts, edges
    if path is None:
        self._set_status("Шлях не знайдено.")
    else:
        self.path_points, self.path_len = path, length
        self._set_status(f"Знайдено! L = {length:.4f}")
    self._redraw()

```

5. ТЕСТУВАННЯ

5.1 Автоматичні тести

Файл `test_algo.py` містить 10 юніт-тестів. Вони не вимагають графічного дисплея і можуть виконуватись у CI-середовищі.

Тест	Що перевіряється
<code>test_segments_cross</code>	Трансверсальний перетин: X-перетин, T-дотик, спільна вершина
<code>test_point_in_polygon</code>	Ray casting для опуклого та неопуклого многокутника
<code>test_visible_no_obstacle</code>	Видимість без перешкод (завжди True)
<code>test_visible_blocked_by_square</code>	Горизонтальний промінь крізь квадрат — не видно
<code>test_visible_adjacent_vertices</code>	Сусідні вершини — видимі; діагональ опуклого полігона — ні
<code>test_shortest_path_no_obstacle</code>	Прямий шлях = евклідова відстань (5.0)
<code>test_shortest_path_simple</code>	Шлях огинає квадрат через кутову вершину; довжина > пряма
<code>test_shortest_path_around_triangle</code>	Обхід трикутника; довжина $\geq$ прямій
<code>test_no_path_inside_obstacle</code>	S всередині перешкоди $\rightarrow$ шлях не знайдено (None)
<code>test_nonconvex_polygon</code>	U-подібна неопукла перешкода — шлях знаходиться

Результат виконання тестів:

```
viktor@ASUS-TUF-Dash-F15-FX517ZC:~/Downloads/ogkg$ python3 test_algo.py
test_segments_cross OK
test_point_in_polygon OK
test_visible_no_obstacle OK
test_visible_blocked_by_square OK
test_visible_adjacent_vertices_of_polygon OK
test_shortest_path_no_obstacle OK
test_shortest_path_simple OK (length=8.3246, direct=8.0000)
test_shortest_path_around_triangle OK (length=10.2462)
test_no_path_inside_obstacle OK
test_nonconvex_polygon OK (length=10.9597, vertices=4)

Всі тести пройдено успішно.
```

Рис. 9. Результати виконання автоматичних тестів – всі 10 тестів пройдено успішно

5.2 Приклади роботи програми

5.2.1 Інтерфейс програми

Після запуску командою `python3 main.py` відкривається інтерактивне вікно: ліворуч — робоча область, праворуч — панель керування з перемикачами режимів та кнопками дій.

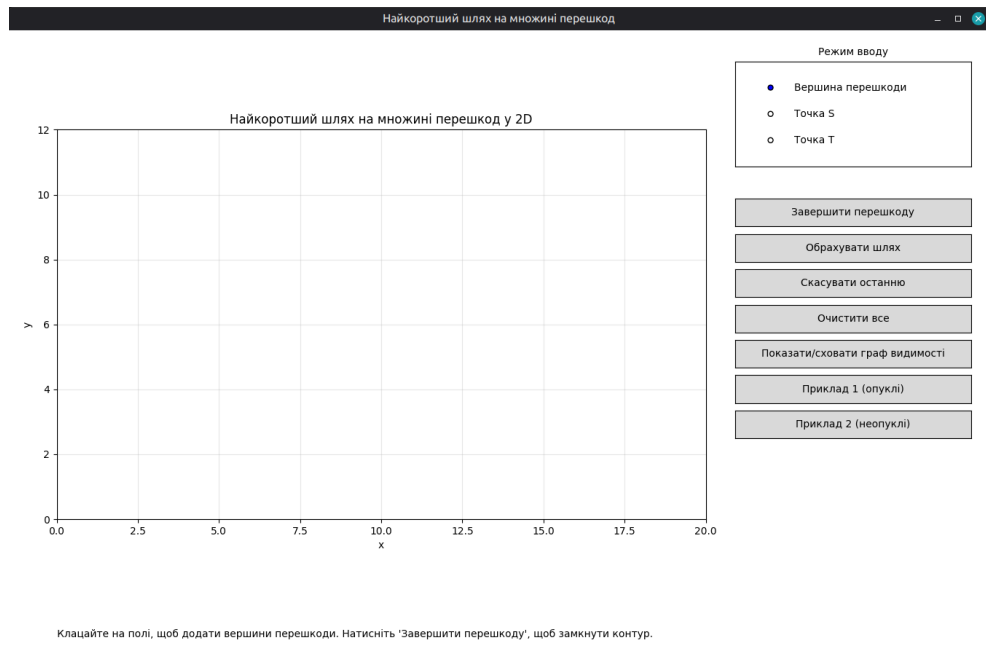
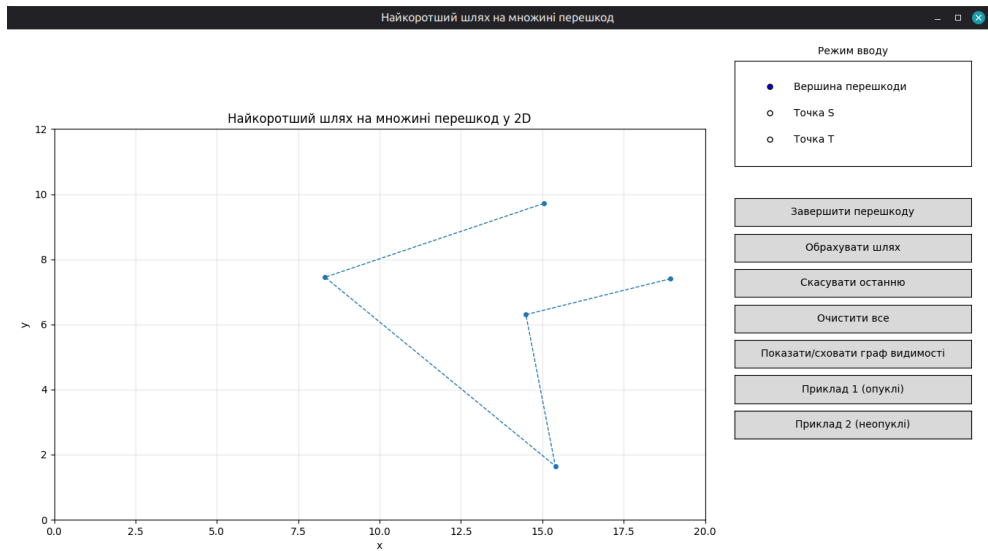


Рис. 1. Інтерфейс програми після запуску

5.2.2 Додавання перешкод

У режимі «Вершина перешкоди» кожен клік лівою кнопкою миші додає точку. Незавершений контур відображається пунктиром. Після натискання «Завершити перешкоду» полігон замикається і заповнюється сірим кольором.



Клацайте на полі, щоб додати вершини перешкоди. Натисніть 'Завершити перешкоду', щоб замкнути контур.

Рис. 2. Процес додавання вершин незавершеної перешкоди (пунктирний контур)

5.2.3 Приклад 1: опуклі перешкоди

Чотири опуклих перешкоди: чотирикутник, трикутник, ромб, малий трикутник. $S = (1.0, 6.0)$, $T = (18.5, 6.0)$. Знайдений шлях — синя ломана, що огинає перешкоди через їхні кутові вершини.

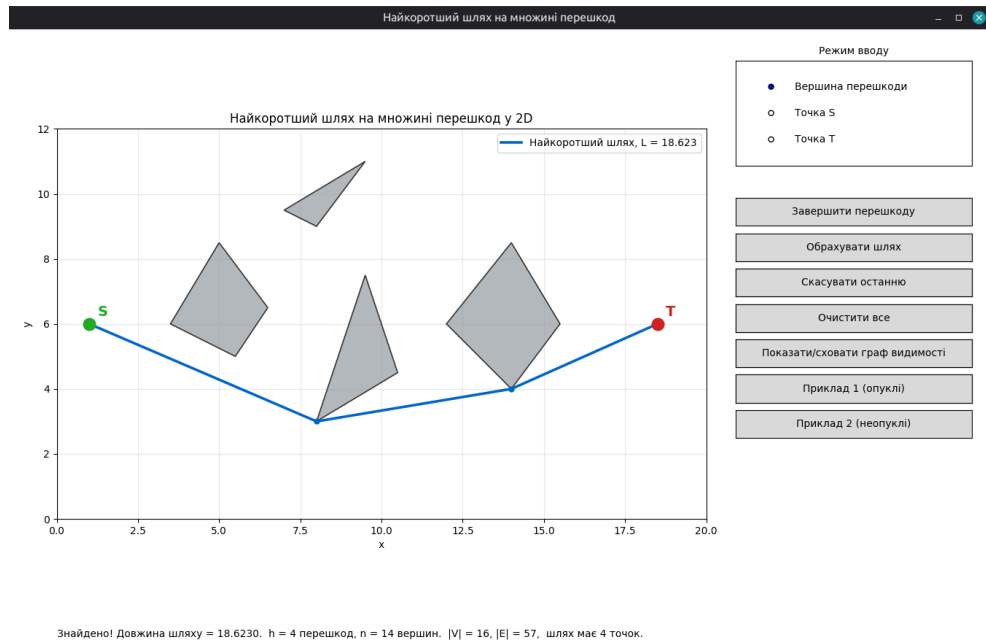


Рис. 3. Приклад 1 – опуклі перешкоди, найкоротший шлях

Увімкнення відображення графа видимості показує всі допустимі з'єднання між вершинами (сірі ребра). Оптимальний шлях — підмножина цих ребер.

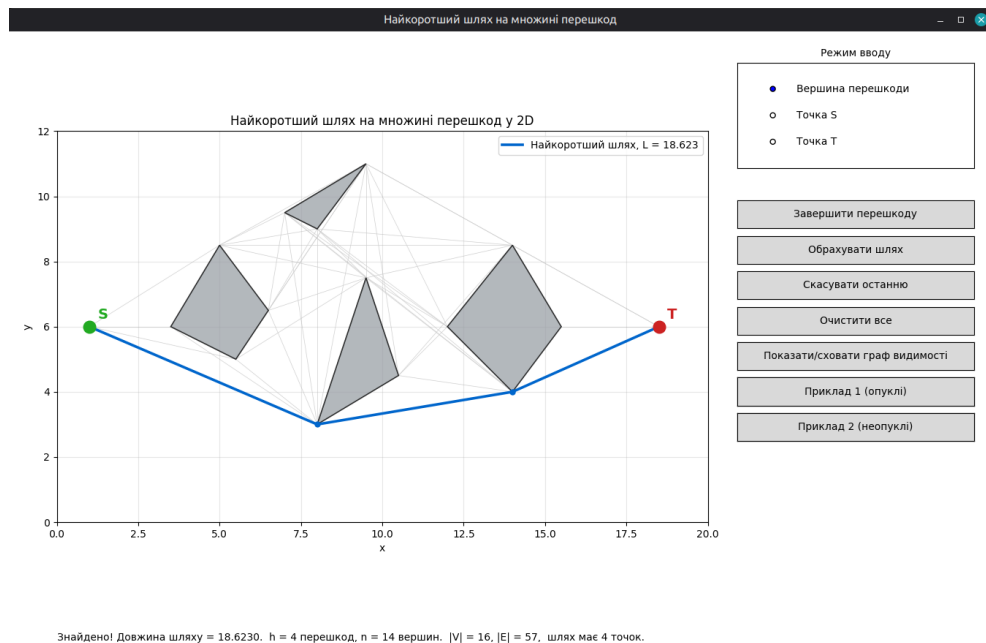


Рис. 4. Приклад 1 – граф видимості (сірі ребра) та оптимальний шлях

5.2.4 Приклад 2: неопуклі перешкоди

U-подібна неопукла перешкода (8 вершин), прямокутник та малий трикутник. Шлях змушений обійти U-форму, що демонструє коректну роботу алгоритму для невипуклих полігонів.

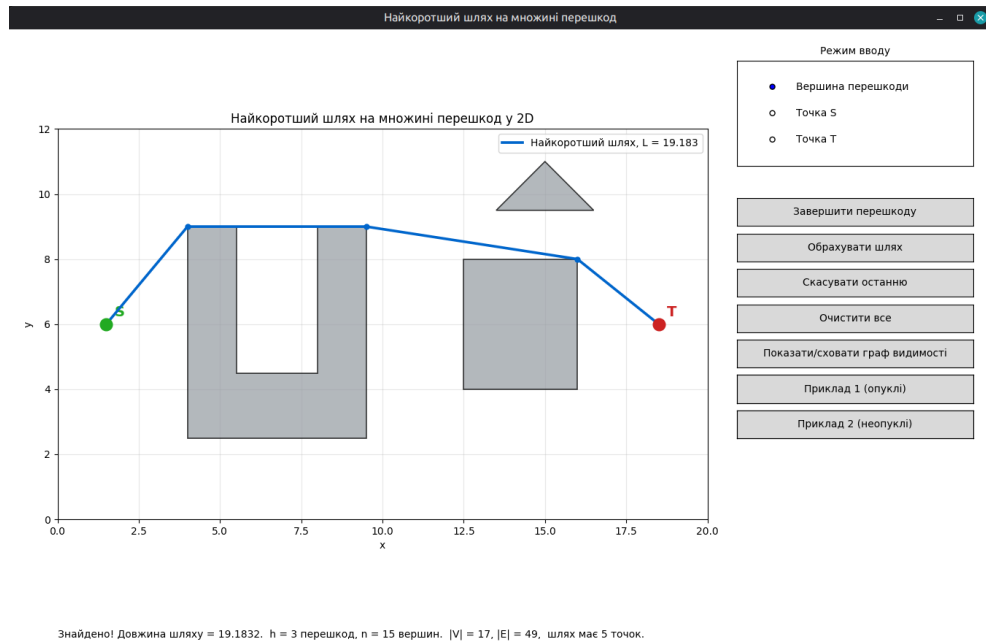


Рис. 5. Приклад 2 – U-подібна неопукла перешкода, шлях огинає її

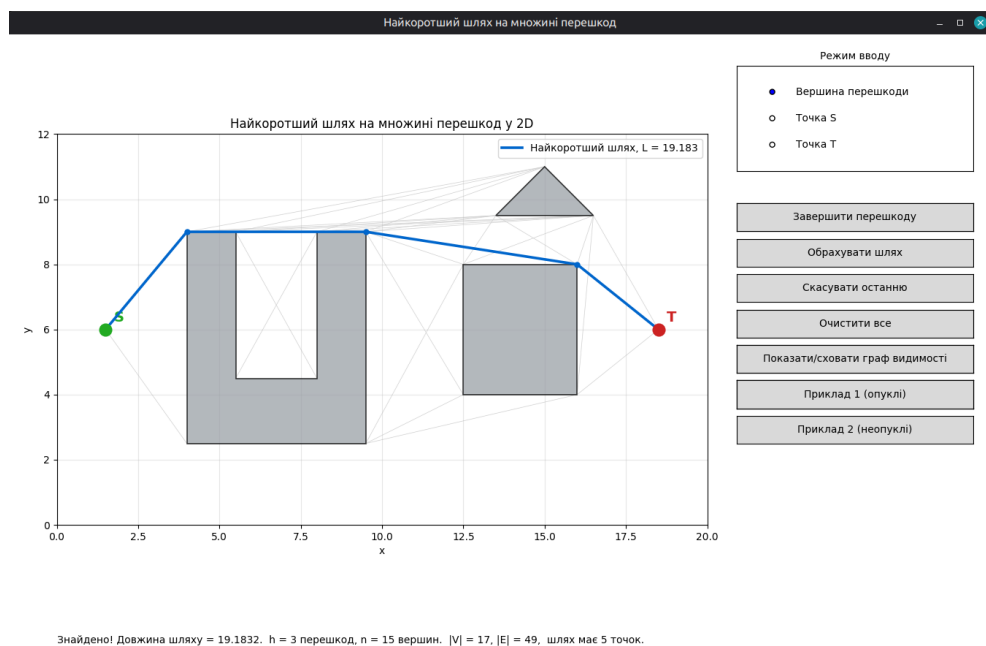


Рис. 6. Приклад 2 – граф видимості для неопуклої конфігурації

5.2.5 Приклад 3: змішані перешкоди

Змішана конфігурація: стрілоподібний неопуклий полігон, ромб та трикутник. Демонструє роботу алгоритму у щільній сцені.

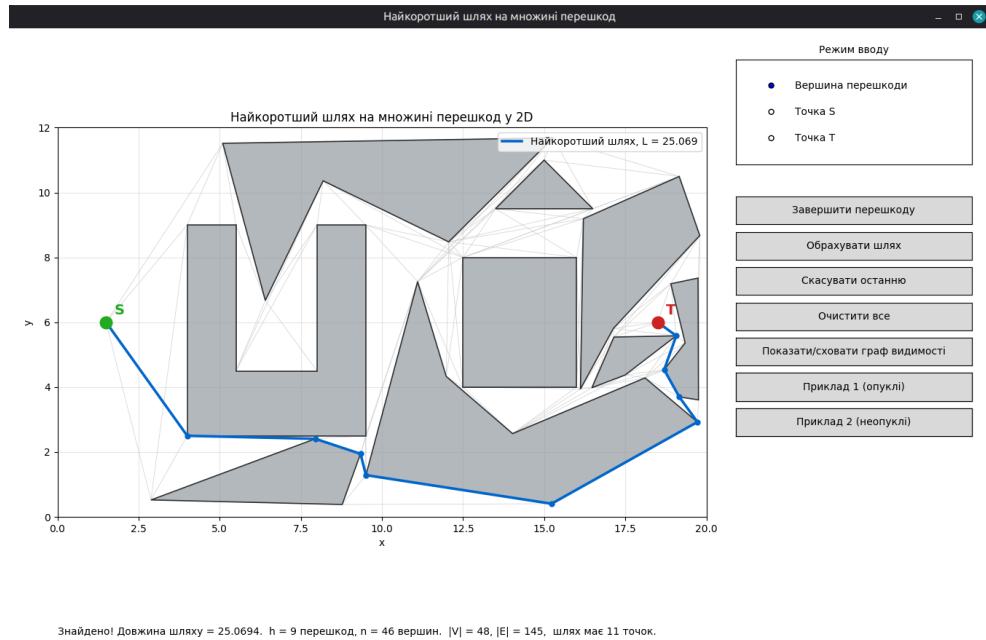


Рис. 7. Приклад 3 – змішані перешкоди, оптимальний шлях

5.2.6 Рядок статусу з результатами

Після обчислення рядок статусу виводить: довжину знайденого шляху, кількість перешкод h , вершин n , розмір графа видимості $|V|$, $|E|$ та кількість точок у шляху.

Знайдено! Довжина шляху = 25.0694. $h = 9$ перешкод, $n = 46$ вершин. $|V| = 48$, $|E| = 145$, шлях має 11 точок.

Рис. 8. Рядок статусу з детальними результатами обчислення

5.2.7 Валідація вхідних даних

Якщо точка S або T розміщена всередині перешкоди, програма виводить повідомлення про помилку і відмовляється від обчислення.

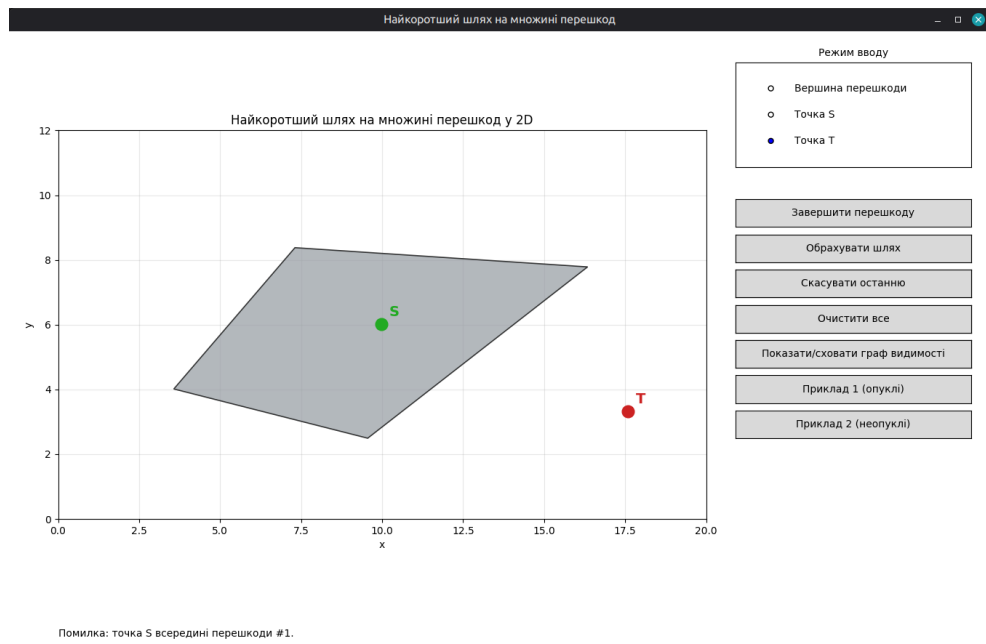


Рис. 10. Валідація: повідомлення про помилку при S всередині перешкоди

6. ВИСНОВКИ

У ході виконання лабораторної роботи реалізовано повноцінну систему пошуку найкоротшого евклідового шляху у 2D-просторі з полігональними перешкодами. Отримані результати:

- Реалізовано набір геометричних примітивів на основі знакового векторного добутку: перевірка трансверсального перетину відрізків, алгоритм Ray Casting.
- Функція `visible()` коректно обробляє опуклі та неопуклі перешкоди, включаючи граничний випадок діагоналі всередині опуклого полігона.
- Граф видимості побудовано наївним алгоритмом $O(N^2 \cdot n)$, прийнятним для задач із кількостями вершин.
- Алгоритм Дейкстри з бінарною купою знаходить оптимальний шлях за $O(N^2 \log N)$.

- Розроблено інтерактивний GUI з підтримкою ручного введення, готових прикладів і відображення графа видимості.
- Всі 10 автоматичних юніт-тестів виконуються успішно.

Можливі напрями вдосконалення: заміна наївної побудови графа алгоритмом Lee & Preparata ($O(n^2 \log n)$); застосування евристики A* замість Дейкстри; підтримка динамічного додавання перешкод без повного перебудовування графа.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Preparata F.P., Shamos M.I. Computational Geometry: An Introduction. Springer-Verlag, 1985. 390 p.
2. de Berg M., Cheong O., van Kreveld M., Overmars M. Computational Geometry: Algorithms and Applications. 3rd ed. Springer, 2008. 386 p.
3. Lee D.T., Preparata F.P. Euclidean shortest paths in the presence of rectilinear barriers. Networks, 14(3):393–410, 1984.
4. Ghosh S.K., Mount D.M. An output-sensitive algorithm for computing visibility graphs. SIAM J. Comput., 20(5):888–910, 1991.
5. Dijkstra E.W. A note on two problems in connexion with graphs. Numerische Mathematik, 1:269–271, 1959.
6. Cormen T.H., Leiserson C.E., Rivest R.L., Stein C. Introduction to Algorithms. 4th ed. MIT Press, 2022. 1312 p.
7. Лекційні матеріали курсу «Обчислювальна геометрія та комп'ютерна графіка» (лекції 6–10: Convex Hull, Visibility Graphs, Proximity).